



Searching and Sorting Algorithms in C

From Linear Scans to Divide-and-Conquer: A Practitioner's
Reference

2026 ACM Handout Series

100 Days Of Code

Abstract: This guide builds searching and sorting from first principles in C — starting with why unsorted data is expensive to query, through the core primitives (swap, comparators, pointer arithmetic), into the canonical algorithms (Linear Search, Binary Search, Bubble/Insertion/Selection Sort, Merge Sort, Quick Sort), their iterative and recursive variants, common bugs, memory behavior on the stack and heap, and a graded practice ladder suitable for competitive programming.

Contents

S.No	Topic	Page
1	Foundations	1
2	Linear Search	2
3	Binary Search	4
4	Bubble Sort	6
5	Insertion Sort	8
6	Selection Sort	10
7	Merge Sort	12
8	Quick Sort	15
9	Competitive Programming Toolkit	18
10	Cumulative Practice Ladder	19
11	Closing	20
12	Appendix A – Debugging Solutions	21
13	Appendix B – Problem Hints	22

1 Foundations

1.1 Why this topic exists

Unsorted data costs $\mathcal{O}(n)$ per lookup — every query scans the whole array. Sorting pays $\mathcal{O}(n \log n)$ once, in exchange for $\mathcal{O}(\log n)$ lookups forever after, plus free deduplication, free min/max, and cheap range queries. The break-even point: sort once + query m times costs $\mathcal{O}(n \log n + m \log n)$ versus $\mathcal{O}(mn)$ unsorted — sorting wins once m is more than a handful.

1.2 Core primitive: swap

C is pass-by-value, so every sort in this handout swaps through pointers:

```

1 void swap(int *a, int *b) {
2     int temp = *a;
3     *a = *b;
4     *b = temp;
5 }
```

1.3 Glossary

Term	Definition
Stability	Equal-key elements keep their original relative order after sorting.
In-place	Sorts using $\mathcal{O}(1)$ – $\mathcal{O}(\log n)$ extra memory, no second array.
Pivot	The element Quick Sort partitions the array around.
Divide and Conquer	Split into independent subproblems, recurse, combine.
Comparator	Function defining ordering between two elements (used by <code>qsort</code>).

1.4 Complexity cheat sheet

Algorithm	Best	Average	Worst	Space	Stable
Linear Search	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	—
Binary Search	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)/\mathcal{O}(\log n)$	—
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Yes
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Yes
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	No
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Yes
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	No

2 Linear Search

2.1 Definition

Scan every element in order until the target is found or the array is exhausted. No structure required; works on unsorted data.

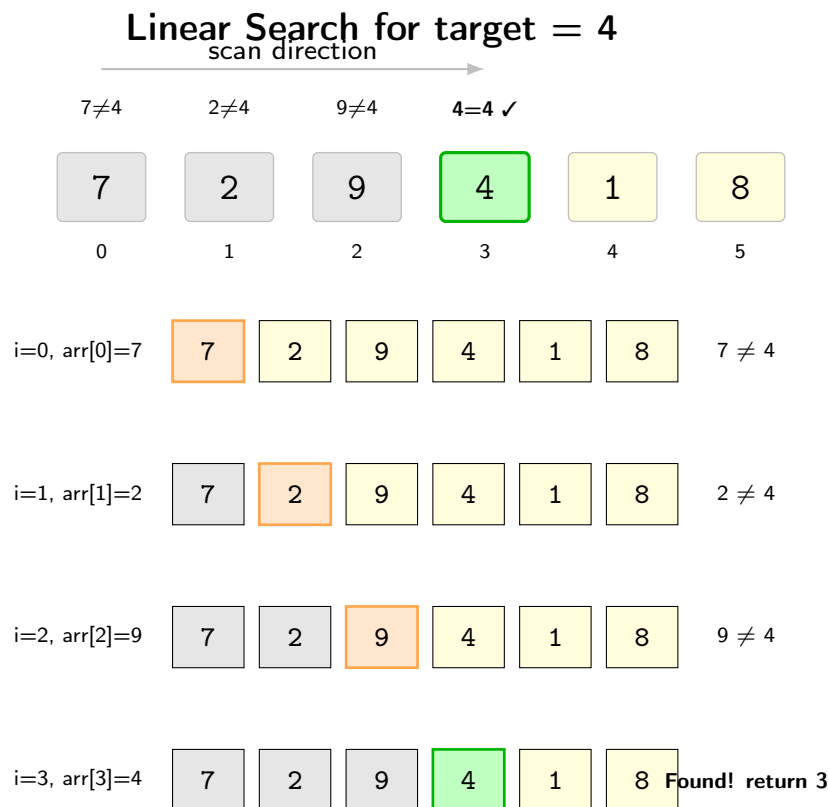


Figure 1: Linear search: scanning left-to-right for target 4 (top), step-by-step walkthrough (below).

2.2 Implementation

```

1 int linear_search(int arr[], int n, int target) {
2     for (int i = 0; i < n; i++) {
3         if (arr[i] == target) return i;
4     }
5     return -1;
6 }
```

2.3 Complexity

$\mathcal{O}(1)$ best (first element), $\mathcal{O}(n)$ average/worst, $\mathcal{O}(1)$ space.

2.4 Analogy

Finding a name in a stack of unsorted business cards — you check every single card until you find the one you want. If the person you're looking for is the first card, great; if they're the last, you go through the whole pile.

Practice Problems

P1. Given `int scores[] = {88, 45, 62, 91, 45, 70};`, write a function that returns the index of the *first* occurrence of 45. Expected output: 1.

P2. Given `int arr[] = {3, 1, 3, 3, 7, 3};`, write `count_occurrences(arr, n, target)` that returns how many times `target` appears using a single linear scan. For `target = 3`, expected output: 4.

Debugging Exercise

```
1 void print_matches(int arr[], int n, int target) {  
2     for (int i = 0; i ≤ n; i++) {  
3         if (arr[i] == target) printf("found at %d\n", i);  
4     }  
5 }
```

Something is wrong with this function. What breaks, and on what input does it break most visibly? (Solution: Appendix A, D1.)

3 Binary Search

3.1 Definition

On a **sorted** array, compare the target to the middle element and discard the half that cannot contain it; repeat on the remaining half.

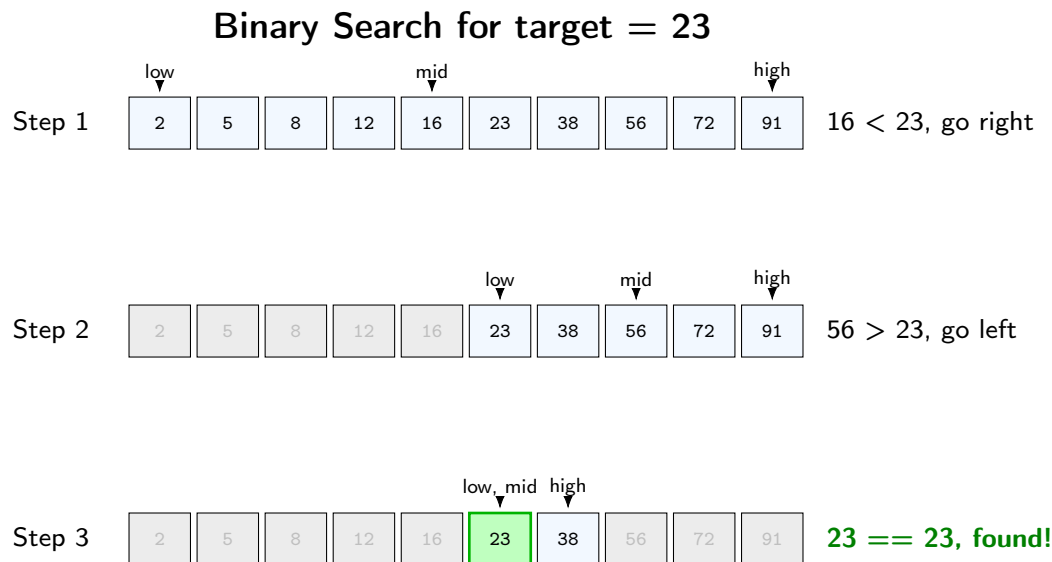


Figure 2: Binary search on `arr = {2,5,8,12,16,23,38,56,72,91}` for target 23, shown in 3 steps.

3.2 Implementation

```

1 // Iterative
2 int binary_search_iterative(int arr[], int n, int target) {
3     int low = 0, high = n - 1;
4     while (low ≤ high) {
5         int mid = low + (high - low) / 2;
6         if (arr[mid] == target) return mid;
7         else if (arr[mid] < target) low = mid + 1;
8         else high = mid - 1;
9     }
10    return -1;
11 }
12
13 // Recursive
14 int binary_search_recursive(int arr[], int low, int high, int target) {
15     if (low > high) return -1;
16     int mid = low + (high - low) / 2;
17     if (arr[mid] == target) return mid;
18     if (arr[mid] < target)
19         return binary_search_recursive(arr, mid + 1, high, target);
20     return binary_search_recursive(arr, low, mid - 1, target);

```

21 }

3.3 Complexity

$\mathcal{O}(\log n)$ always. $\mathcal{O}(1)$ space iterative, $\mathcal{O}(\log n)$ recursive (call stack).

3.4 Analogy

Finding a name in an alphabetized phone book — open to the middle, decide “earlier or later,” tear that half in half again, and repeat. You never check every page; four halvings reduce 16 pages to 1.

Practice Problems

P3. `arr = {2,5,8,12,16,23,38,56,72,91}`, find `target = 23`. Expected output: index 5.

P4. `arr = {1,2,2,2,3,4,5}` (has duplicates). Write `first_occurrence` and `last_occurrence` functions using modified binary search. For `target = 2`, expected: first = 1, last = 3.

P5. `arr = {4,5,6,7,0,1,2}` is a *rotated* sorted array. Write `search_rotated(arr, n, target)` that still runs in $\mathcal{O}(\log n)$. For `target = 0`, expected output: index 4. (Hint: at every step, one half of the current range is guaranteed to be normally ordered — check which.)

Debugging Exercise

D2.

```
1 while (low ≤ high) {
2     int mid = low + (high - low) / 2;
3     if (arr[mid] > target) high = mid;
4     else low = mid + 1;
5 }
```

Trace this by hand on `arr = {1,2,3}`, `target = 10`. What happens, and why?

D3.

```
1 int binary_search_recursive_buggy(int arr[], int low, int high, int
   target) {
2     int mid = low + (high - low) / 2;
3     if (arr[mid] == target) return mid;
4     if (arr[mid] < target)
5         return binary_search_recursive_buggy(arr, mid + 1, high,
           target);
6     return binary_search_recursive_buggy(arr, low, mid - 1, target);
7 }
```

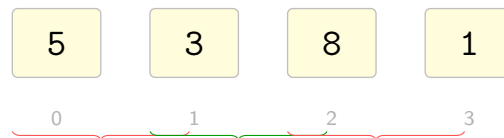
Call this with `low = 0`, `high = -1` (i.e. an empty range, or a target not in the array). What happens? (Solutions: Appendix A, D2–D3.)

4 Bubble Sort

4.1 Definition

Repeatedly walk the array, swapping adjacent out-of-order pairs, so the largest unsorted element “bubbles” to its correct position each pass.

Pass 1 — comparing adjacent pairs



5 > 3, swap 3 < 8, ok 8 > 1, swap

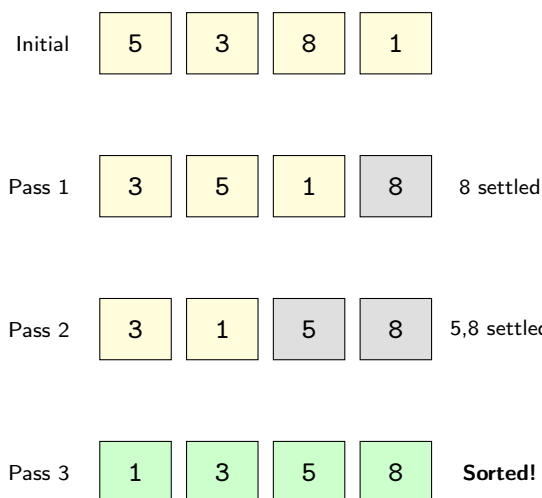


Figure 3: Bubble sort: adjacent-pair comparisons in a single pass (top) and full pass-by-pass walk-through (below).

4.2 Implementation

```

1 void bubble_sort(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         int swapped = 0;
4         for (int j = 0; j < n - i - 1; j++) {
5             if (arr[j] > arr[j + 1]) {
6                 swap(&arr[j], &arr[j + 1]);
7                 swapped = 1;
8             }
9         }
10        if (!swapped) break;    // already sorted
11    }
12 }
```


4.3 Complexity

$\mathcal{O}(n)$ best (already sorted, early exit), $\mathcal{O}(n^2)$ average/worst, $\mathcal{O}(1)$ space, stable.

4.4 Analogy

Bubbles rising in a glass of soda — the largest bubbles reach the top first. In a tray of muddy water, the heaviest particles sink to the bottom fastest. Each pass lets the next-biggest element settle, like one bubble popping at the surface at a time.

Practice Problems

P6. `arr = {30, 10, 20}`. Modify `bubble_sort` to also return the total number of swaps performed. Expected: sorted array `{10, 20, 30}`, swap count 2.

Debugging Exercise

```
1 void bubble_sort_buggy(int arr[], int n) {  
2     for (int i = 0; i < n - 1; i++) {  
3         for (int j = 0; j < n - i - 1; j++) {  
4             if (arr[j] < arr[j + 1]) { // note the direction  
5                 swap(&arr[j], &arr[j + 1]);  
6             }  
7         }  
8     }  
9 }
```

The function compiles and runs without crashing on any input. What does it actually produce, and why might that be an easy mistake to miss in review? (Solution: Appendix A, D4.)

5 Insertion Sort

5.1 Definition

Maintain a growing sorted prefix; take the next element and shift it leftward into its correct position within that prefix.

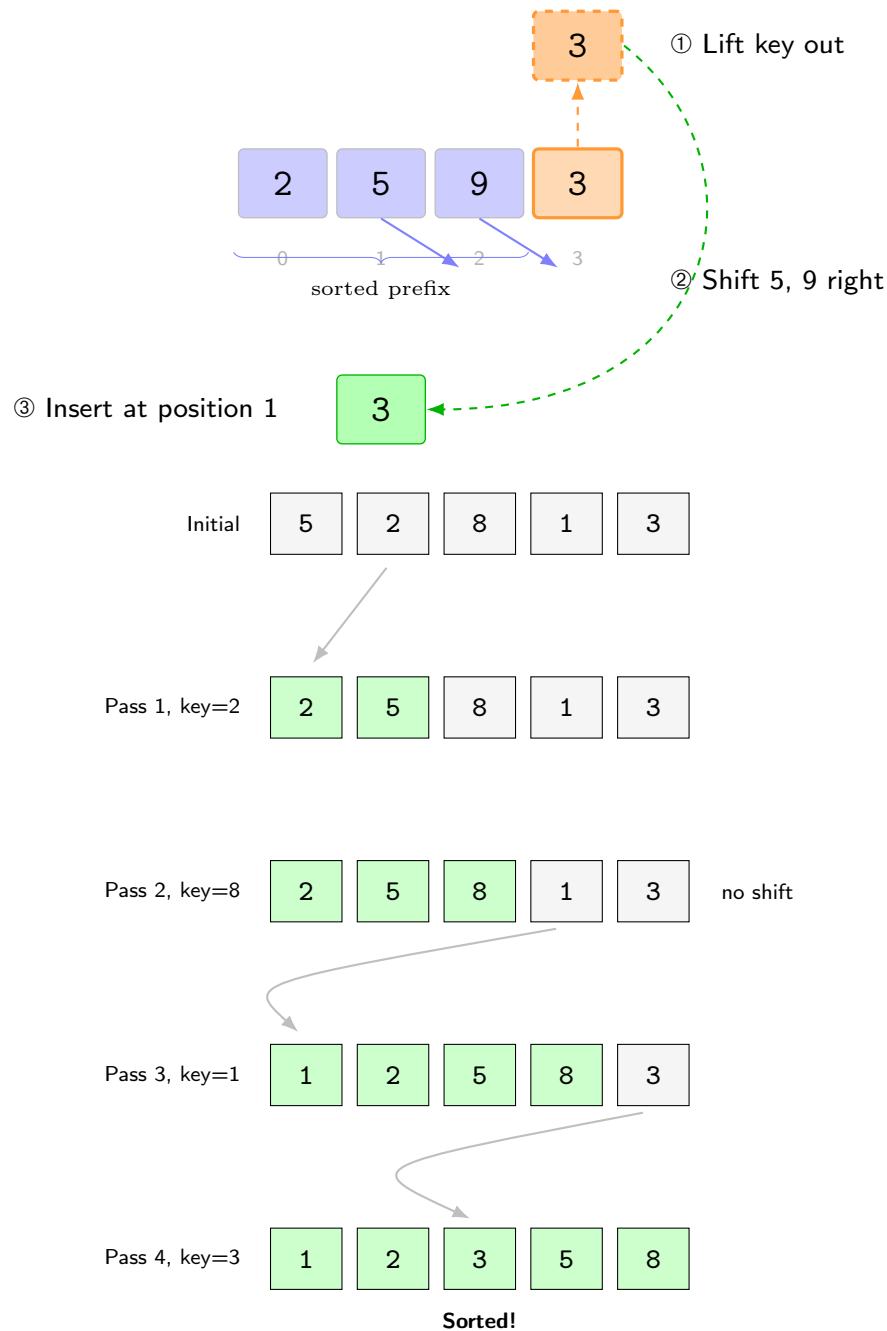


Figure 4: Insertion sort: key element lifted and inserted (top), pass-by-pass walkthrough (below).

5.2 Implementation

```

1 void insertion_sort(int arr[], int n) {
2     for (int i = 1; i < n; i++) {
3         int key = arr[i];
4         int j = i - 1;
5         while (j ≥ 0 && arr[j] > key) {

```

```

6         arr[j + 1] = arr[j];
7         j--;
8     }
9     arr[j + 1] = key;
10 }
11 }

```

5.3 Complexity

$\mathcal{O}(n)$ best (nearly sorted — this is why it's the go-to for small/nearly-sorted data), $\mathcal{O}(n^2)$ average/worst, $\mathcal{O}(1)$ space, stable.

5.4 Analogy

Organizing a hand of playing cards — pick up cards one at a time and insert each into its correct position among the cards already in your hand. A nearly sorted hand (one card out of place) takes almost no work; a fully shuffled hand takes much longer.

Practice Problems

P7. Compare shift counts: run `insertion_sort` on $\{1, 2, 3, 4, 9\}$ (only last element out of place) versus $\{9, 4, 3, 2, 1\}$ (fully reversed), both size 5. Instrument the function to count total shift operations. Which one does far fewer shifts, and does that match the $\mathcal{O}(n)$ best-case claim?

Debugging Exercise

```

1 void insertion_sort_buggy(int arr[], int n) {
2     for (int i = 1; i < n; i++) {
3         int key = arr[i];
4         int j = i - 1;
5         while (j ≥ 0 && arr[j] > key) {
6             arr[j + 1] = arr[j];
7         }
8         arr[j + 1] = key;
9     }
10 }

```

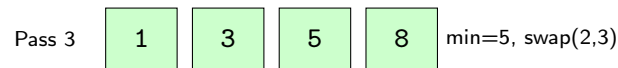
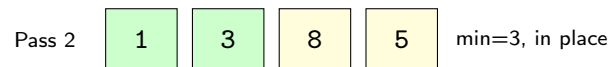
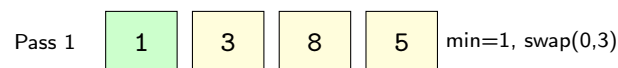
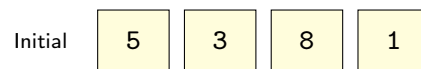
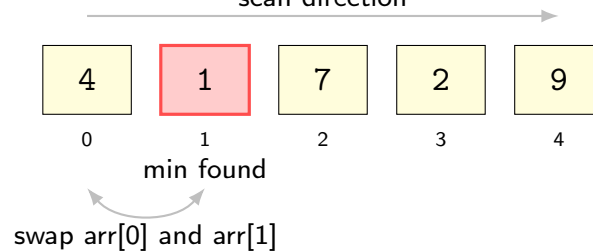
Run this (carefully, with a debugger or on paper) on `arr = {5, 3, 8}`. What happens on the very first outer iteration? (Solution: Appendix A, D5.)

6 Selection Sort

6.1 Definition

Find the minimum of the unsorted remainder and swap it into the next position, growing a sorted prefix from the front.

Pass 1 — scan unsorted region for minimum



Sorted!

Figure 5: Selection sort: scanning for minimum then swapping (top), pass-by-pass walkthrough (below).

6.2 Implementation

```

1 void selection_sort(int arr[], int n) {
2     for (int i = 0; i < n - 1; i++) {
3         int min_idx = i;
4         for (int j = i + 1; j < n; j++) {
5             if (arr[j] < arr[min_idx]) min_idx = j;
6         }
7         if (min_idx != i) swap(&arr[i], &arr[min_idx]);
8     }
9 }

```

6.3 Complexity

$\mathcal{O}(n^2)$ in all cases (it always scans the full remainder regardless of order), $\mathcal{O}(1)$ space, **not** stable.

6.4 Analogy

Finding the shortest person in a line and moving them to the front, then finding the next-shortest among the rest and moving them to position two, and so on — each pass does a full scan of the remaining line whether the line is well-ordered or completely random.

Practice Problems

P8. Modify `selection_sort` to sort **descending** by finding the maximum of the unsorted remainder each pass instead of the minimum. Test on `{4,1,7,2,9}`; expected: `{9,7,4,2,1}`.

Debugging Exercise

```
1 void selection_sort_buggy(int arr[], int n) {  
2     for (int i = 0; i < n - 1; i++) {  
3         int min_idx = i;  
4         for (int j = i + 1; j < n; j++) {  
5             if (arr[j] < arr[min_idx]) {  
6                 swap(&arr[i], &arr[j]);    // swaps immediately  
7                 min_idx = j;  
8             }  
9         }  
10    }  
11 }
```

Trace this on `arr = {5, 1, 4, 2}` by hand, one comparison at a time. Does the final array end up sorted? (Solution: Appendix A, D6.)

7 Merge Sort

7.1 Definition

Divide & Conquer: split the array in half recursively until each piece is trivially sorted (size 1), then merge sorted halves back together.

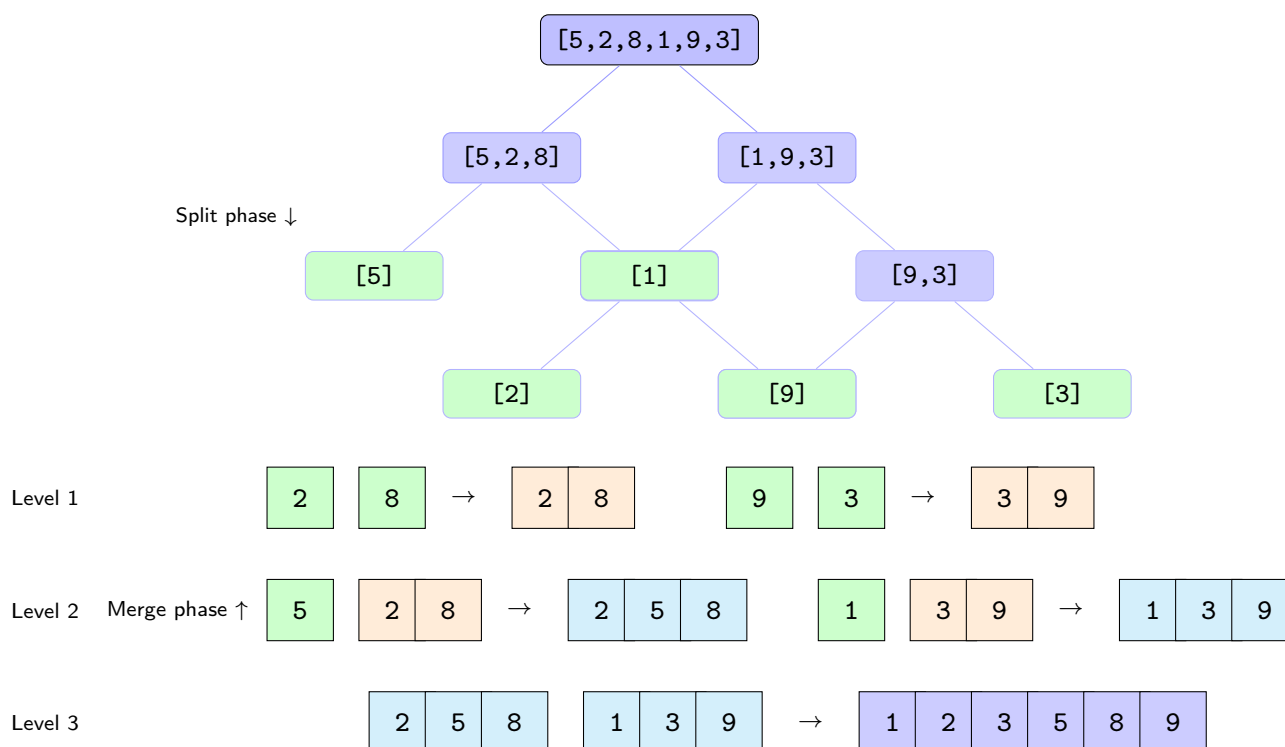


Figure 6: Merge sort: recursive split phase (top) and bottom-up merge phase (below).

7.2 Implementation

```

1 void merge(int arr[], int left, int mid, int right) {
2     int n1 = mid - left + 1;
3     int n2 = right - mid;
4
5     int *L = (int *)malloc(n1 * sizeof(int));
6     int *R = (int *)malloc(n2 * sizeof(int));
7
8     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
9     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
10
11     int i = 0, j = 0, k = left;
12     while (i < n1 && j < n2) {
13         arr[k++] = (L[i] ≤ R[j]) ? L[i++] : R[j++];
14     }
15     while (i < n1) arr[k++] = L[i++];
16     while (j < n2) arr[k++] = R[j++];
17 }

```

```

18     free(L);
19     free(R);
20 }
21
22 // Recursive (top-down)
23 void merge_sort(int arr[], int left, int right) {
24     if (left < right) {
25         int mid = left + (right - left) / 2;
26         merge_sort(arr, left, mid);
27         merge_sort(arr, mid + 1, right);
28         merge(arr, left, mid, right);
29     }
30 }
31
32 // Iterative (bottom-up)
33 void merge_sort_iterative(int arr[], int n) {
34     for (int width = 1; width < n; width *= 2) {
35         for (int left = 0; left < n - 1; left += 2 * width) {
36             int mid = (left + width - 1 < n - 1)
37                 ? left + width - 1 : n - 1;
38             int right = (left + 2 * width - 1 < n - 1)
39                 ? left + 2 * width - 1 : n - 1;
40             if (mid < right) merge(arr, left, mid, right);
41         }
42     }
43 }

```

7.3 Complexity

$\mathcal{O}(n \log n)$ in all cases, $\mathcal{O}(n)$ auxiliary space (the L/R buffers), stable. The $\mathcal{O}(n)$ space comes from `merge()` needing temporary copies — merging in place without extra space would force $\mathcal{O}(n)$ shifting per merge, destroying the time bound.

7.4 Analogy

Splitting a pile of exams between two graders, who each split their pile, down to single exams — everyone grades one exam instantly, then pairs of graders merge their sorted results back together. Splitting takes no work; all the effort is in the merging phase.

Practice Problems

P9. Write a standalone `merge_two_sorted(a, na, b, nb, out)` using just the merge logic (no recursion) to combine `a = {1,3,5}` and `b = {2,4,6}` into `out = {1,2,3,4,5,6}`.

P10. (Classic CP problem) Count the number of **inversions** in `arr = {2,4,1,3,5}` (a pair $i < j$ where $a[i] > a[j]$) by modifying `merge()` to tally a count every time an element from the right half is placed before remaining elements of the left half. Expected output: 3 (pairs: (4,1), (4,3), (2,1)).

Debugging Exercise

D7.

```
1 while (i < n1 && j < n2) {
2     arr[k++] = (L[i] < R[j]) ? L[i++] : R[j++];    // note: < not ≤
3 }
```

This never crashes and produces a correctly *sorted* array on every input. What guarantee does it silently violate, and on what kind of input would you actually observe the difference?

D8.

```
1 void merge_sort_buggy(int arr[], int left, int right) {
2     if (left < right) {
3         int mid = (left + right) / 2;
4         merge_sort_buggy(arr, left, right);    // look closely
5         merge_sort_buggy(arr, mid + 1, right);
6         merge(arr, left, mid, right);
7     }
8 }
```

What happens when you call this on an array of size ≥ 2 ? (Solutions: Appendix A, D7–D8.)

8 Quick Sort

8.1 Definition

Choose a pivot, partition the array so everything less than the pivot is to its left and everything greater is to its right (the pivot lands in its final position), then recurse on each side independently.

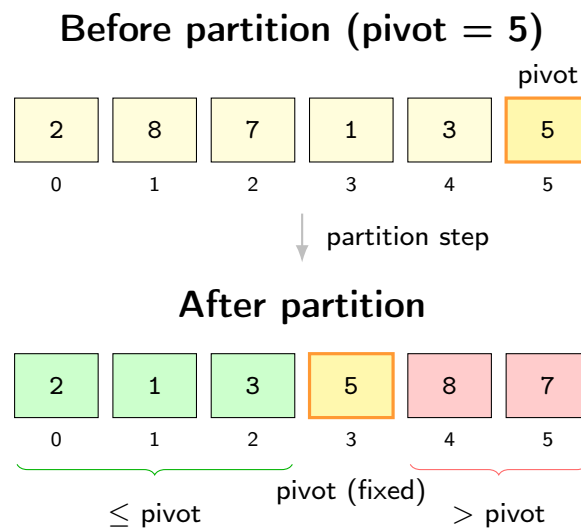


Figure 7: Quick sort partition: before and after, with pivot regions labelled.

8.2 Implementation

```

1  int partition(int arr[], int low, int high) {
2      int pivot = arr[high];
3      int i = low - 1;
4      for (int j = low; j < high; j++) {
5          if (arr[j] ≤ pivot) {
6              i++;
7              swap(&arr[i], &arr[j]);
8          }
9      }
10     swap(&arr[i + 1], &arr[high]);
11     return i + 1;
12 }

13
14 // Recursive
15 void quick_sort(int arr[], int low, int high) {
16     if (low < high) {
17         int pi = partition(arr, low, high);
18         quick_sort(arr, low, pi - 1);
19         quick_sort(arr, pi + 1, high);
20     }

```

```

21 }
22
23 // Iterative, explicit stack
24 void quick_sort_iterative(int arr[], int low, int high) {
25     int size = high - low + 1;
26     int *stack = (int *)malloc(size * sizeof(int));
27     int top = -1;
28     stack[++top] = low;
29     stack[++top] = high;
30
31     while (top ≥ 0) {
32         high = stack[top--];
33         low = stack[top--];
34         int pi = partition(arr, low, high);
35         if (pi - 1 > low) { stack[++top] = low;    stack[++top] = pi -
            1; }
36         if (pi + 1 < high) { stack[++top] = pi + 1; stack[++top] = high;
            }
37     }
38     free(stack);
39 }

```

8.3 Why it usually beats Merge Sort in practice

Quick Sort sorts in place, repeatedly touching contiguous memory near the pivot, so it stays cache-friendly. Merge Sort allocates and copies into separate buffers at every merge, costing more memory traffic even with a similar comparison count. Production sorts (introsort) hedge against Quick Sort's $\mathcal{O}(n^2)$ worst case by randomizing the pivot or falling back to Heap Sort past a recursion-depth threshold.

8.4 Complexity

$\mathcal{O}(n \log n)$ best/average, $\mathcal{O}(n^2)$ worst (adversarial pivot choice, e.g. already-sorted input with last-element pivoting), $\mathcal{O}(\log n)$ stack space, **not** stable.

8.5 Analogy

Sorting a group of students by height — pick one student as a reference, move everyone shorter to the left and everyone taller to the right, then repeat independently on each group. The reference student stays put forever. If you pick the tallest student as the reference, everyone goes to the left and you gain nothing (the worst case).

Practice Problems

P11. (QuickSelect) `arr = {3,2,1,5,6,4}`, find the **2nd largest** element without fully sorting — partition like Quick Sort but recurse into only the side guaranteed to contain the answer. Expected output: 5.

P12. `arr = {4,4,4,2,4,4,1,4}` has many duplicate values. Implement a **three-way partition** (Dutch National Flag: $< \text{pivot}$, $= \text{pivot}$, $> \text{pivot}$ in one pass) and compare its behavior to the standard Lomuto partition on this input.

Debugging Exercise

D9.

```
1 int partition_buggy(int arr[], int low, int high) {
2     int pivot = arr[high];
3     int i = low - 1;
4     for (int j = low; j ≤ high; j++) {    // note: ≤ high
5         if (arr[j] ≤ pivot) {
6             i++;
7             swap(&arr[i], &arr[j]);
8         }
9     }
10    return i;
11 }
```

Run this on `arr = {3,1,2}` (`low=0`, `high=2`) by hand. Where does the pivot end up, and what does `partition_buggy` return versus what the caller expects?

D10. The *correct* `partition()` from Section 7.3 is used to sort `arr = {5,5,5,5,5,5,5,5}` (8 identical elements). It doesn't crash and it doesn't produce a wrong answer — but it's dramatically slower than you'd expect for size 8. Why, and what's the fix? (Solutions: Appendix A, D9–D10.)

9 Competitive Programming Toolkit

9.1 qsort and comparators

```
1 int compare_ints(const void *a, const void *b) {  
2     int x = *(const int *)a;  
3     int y = *(const int *)b;  
4     return (x > y) - (x < y);    // never 'return x - y;' -- overflows  
5 }  
6 // qsort(arr, n, sizeof(int), compare_ints);
```

`qsort` is **not** guaranteed stable — don't rely on tie-break order without an explicit secondary key.

9.2 Platform tips

- `mid = low + (high - low) / 2`, never `(low + high) / 2` — sidesteps overflow.
- Randomize the pivot (or use median-of-three) before partitioning if adversarial input is plausible.
- For arrays with many duplicates, use three-way partitioning (see D10) to avoid $\mathcal{O}(n^2)$ degeneration.
- Read large inputs with `scanf` loops over per-token manual parsing.
- Prefer `qsort()` over hand-rolled sorts unless the problem tests sort implementation.

9.3 Common mistakes checklist

- ☐ Forgot to pass array length alongside the pointer.
- ☐ `(low + high) / 2` instead of the overflow-safe form.
- ☐ Comparator subtracts `(a - b)` instead of comparing explicitly.
- ☐ Assumed `qsort()` is stable.
- ☐ Off-by-one loop bound (`<=` vs `<`).
- ☐ No duplicate-handling strategy, causing degenerate Quick Sort behavior.
- ☐ Passed a `struct` wrapping an array by value instead of by pointer, silently sorting a copy.

10 Cumulative Practice Ladder

Level	Problem	Technique
1	Binary search a sorted array	Two-pointer halving
1	Implement any $\mathcal{O}(n^2)$ sort from scratch	Nested loop + swap
2	Sort via Merge Sort or Quick Sort	Divide & Conquer
2	Two-Sum on a sorted array	Two pointers from both ends
3	Search in a rotated sorted array	Modified binary search
3	k -th largest element	QuickSelect

11 Closing

You now have, per algorithm, a working definition, a diagram, tested C code, and its complexity — plus 12 practice problems and 10 debugging exercises to actually exercise the material instead of just reading about it.

Natural next steps: **Hash Tables** for $\mathcal{O}(1)$ average lookup when order doesn't matter, and **Trees/Heaps** for structures that stay sorted-ish under insertion and deletion, which a plain array cannot do efficiently.

12 Appendix A – Debugging Exercise Solutions

D1. `for (int i = 0; i <= n; i++)` reads `arr[n]`, one past the end — out-of-bounds. Most visible when `target` hits whatever garbage sits at `arr[n]`, producing a phantom match. Fix: `i < n`.

D2. On `{1,2,3}`, target 10: all comparisons false, `low` increments toward `high`, this specific case terminates. Flip it: on `{10,20,30}` searching for 5, `arr[mid] > target` is always true so `high = mid` forever, infinite loop. Fix: `high = mid - 1`.

D3. With `low = 0`, `high = -1`, no `if (low > high) return -1;` guard, so `mid` is invalid and `arr[mid]` reads out of bounds. Fix: add the base case as the first line.

D4. The comparison `arr[j] < arr[j+1]` sorts into **descending** order — the opposite of what bubble sort normally means. Doesn't crash, still "sorts," just the wrong direction.

D5. The `while` loop never decrements `j`, so once `arr[j] > key` is true it stays true forever — infinite loop. Fix: add `j--;` inside the loop body.

D6. Swapping immediately (rather than tracking `min_idx`) moves the smaller value into position but then `min_idx` points at the wrong index holding the old larger value. Trace `{5,1,4,2}`: after the swap, `min_idx = 1` but position 1 now holds 5, not 1. Not correctly sorted.

D7. Using `<` instead of `≤` breaks **stability**: when `L[i] == R[j]`, the right-half element goes first. Array is still numerically correct; you'd only notice with duplicate-key payload data.

D8. The first recursive call passes `(arr, left, right)` — identical arguments each time — recursing forever. Fix: `merge_sort_buggy(arr, left, mid);`.

D9. `arr = {3,1,2}`, `pivot = 2`. The loop includes `j = high`, comparing the pivot to itself, incrementing `i` and performing a self-swap. Returns `i = 1` but pivot is at index 2 — caller believes pivot's position is 1, which is wrong, corrupting subsequent recursive calls.

D10. All 8 elements equal, every `<=` is true, pivot lands at `high` every time — the degenerate already-sorted case, $\mathcal{O}(n^2)$. Fix: three-way (Dutch National Flag) partition that excludes equal elements from further recursion.

13 Appendix B – Practice Problem Hints

#	Hint
P1	Return the index the moment you find a match — don't keep scanning.
P2	Increment a counter instead of returning on first match.
P3	Standard iterative binary search; verify <code>low <= high</code> as the loop condition.
P4	For “first occurrence,” when you find a match keep searching the <i>left</i> half; for “last occurrence,” keep searching the <i>right</i> half.
P5	At each step, check whether <code>arr[low..mid]</code> or <code>arr[mid..high]</code> is normally sorted, then check if <code>target</code> falls inside that sorted half's value range.
P6	Add a counter variable, increment it inside the <code>if (arr[j] > arr[j+1])</code> block.
P7	Add a counter inside the <code>while</code> loop of insertion sort; compare totals across both inputs.
P8	Flip the comparison in the inner loop from <code><</code> to <code>></code> , and rename <code>min_idx</code> conceptually to <code>max_idx</code> .
P9	This is exactly the <code>merge()</code> helper's inner loop, just without the surrounding recursive calls.
P10	Every time you take from the right half while left-half elements remain, add the count of remaining left-half elements to your inversion total.
P11	Partition once; if the pivot's final index matches the target rank, you're done – otherwise recurse into only the side that must contain it.
P12	Use two extra pointers (<code>lt</code> , <code>gt</code>) alongside the scan pointer to maintain three regions in a single pass.